Trees, Bagging, and Multiclass Boosting on Olivetti Faces

Jiya Yadav
Roll number: M26CSE004

Submission repository: included in Jiya_Yadav_M26CSE004_Assignment2.zip
Code snapshot: 0679161a6a89

October 19, 2026

Abstract

I studied decision trees, bootstrap bagging, and SAMME boosting on the 40-person Olivetti faces data. The split was made separately inside each identity, so every person contributed six training images, two validation images, and two test images. A single unpruned tree reached 53.75% clean test accuracy. Bagging 100 copies of that tree raised it to 85.00% and remained much stronger after the centre of each test face was removed. Boosting behaved differently: it kept returning to a small set of hard training faces, but the selected model did not match bagging on this data. Thirty resampled training sets make the difference clear. Bagging sharply reduced variance while keeping useful depth; a shallow tree had little variance to remove but too much bias.

1 Data and common setup

The seed is 26004, obtained by reading the numeric characters of M26CSE004 in order. I used fetch_olivetti_faces without shuffling the source data, then applied one seeded permutation inside each identity. The saved split contains 240 training, 80 validation, and 80 test images. Inputs are the raw 64×64 pixels flattened to 4096 numbers. No scaling, PCA, or feature selection was used. For the second test set I set rows 24 to 39 and columns 24 to 39 to zero. Every model was trained on clean faces only.

All choices were made from clean validation accuracy. When settings tied exactly, I used the smaller ensemble and the less fragile tree setting. The test set and its masked copy were opened only after the choices were frozen. Labels below are the integer identities 0 through 39 returned by scikit-learn.

Table 1: Tree results for min_samples_leaf=2. Accuracies are percentages. $\bar{n}$ is the mean number of training examples in an occupied leaf and f_1 is the fraction of leaves containing one training example.

Depth	Train	Val.	Test	Leaves	$\bar{n}$	f_1
2	7.50	6.25	7.50	3	80.00	0
4	12.50	8.75	8.75	5	48.00	0
8	22.50	16.25	18.75	9	26.67	0
16	44.58	23.75	28.75	18	13.33	0
None	97.08	51.25	53.75	47	5.11	0

2 Single tree and bagging

2.1 Choosing the tree

I trained all 15 requested depth and leaf-size combinations. Table 1 shows the leaf-size-two row at every depth because it also contains the selected setting; the complete grid is saved as results/tree_grid.csv. Leaf sizes one, two, and four were identical through depth 16. At unlimited depth, leaf sizes one and two tied at 51.25% validation accuracy, while leaf size four fell to 47.50%. I kept leaf size two because it removed the singleton leaves without lowering validation accuracy.

Depth changes what the tree can separate. The depth-two tree makes only three occupied leaves for 40 identities, so most classes are forced together. More depth steadily lowers this restriction, but it also makes smaller leaves. The unlimited leaf-size-one tree went all the way to 100% training accuracy and had 14.89% singleton leaves. Leaf size two gave nearly the same fit without those one-image rules.

2.2 Bagging from bootstrap samples

For each tree I drew 240 training positions with replacement and fit the frozen tree setting. Prediction is an unweighted class vote. Table 2 uses prefixes of the same 200-tree sequence. A bootstrap sample contained about 152 distinct faces, almost exactly the

Table 2: Bagging results. U is the mean number of distinct faces in a bootstrap sample.

B	Train	Val.	Test	U	OOB
1	67.92	43.75	46.25	136.00	30.77
5	92.92	62.50	66.25	150.60	28.50
10	99.58	73.75	73.75	152.10	41.35
25	100	87.50	83.75	151.76	60.42
50	100	85.00	86.25	152.54	71.67
100	100	88.75	85.00	152.18	75.42
200	100	88.75	85.00	152.15	78.33

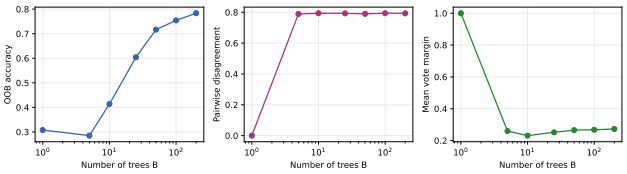


Figure 1: OOB accuracy, validation-set tree disagreement, and validation vote margin for bagging prefixes.

finite-sample expectation

$$240\left[1-\left(1-\frac{1}{240}\right)^{240}\right]=151.89. \tag{1}$$

The 100-tree and 200-tree models tied on validation accuracy, so I froze $B = 100$. The first OOB figure is noisy because only 43.33% of training faces receive an OOB vote at $B = 1$. Coverage reaches 100% at $B = 25$.

Figure 1 explains why adding trees first helps and then settles. Pairwise disagreement stays close to 0.79. This is useful diversity rather than a sign that the vote is failing: the mean margin rises from 0.23 at ten trees to 0.27 at 100, while OOB accuracy keeps improving. After 100 trees, the validation prediction no longer changes.

2.3 Clean and masked faces

The single tree is very sure even when it is wrong. Masking cuts its accuracy by 35 points while its mean confidence rises to one, an effect of landing in pure terminal leaves. Bagging loses 16.25 points and its vote becomes less concentrated, which is a more honest response to missing pixels.

On test faces 8 and 14 (source indices), the tree sends both examples to class 35. Bootstrap trees split across several plausible classes, but the correct classes 0 and 1 win with 15 and 18 votes. This is the kind of unstable boundary error that averaging can fix. For source indices 6 and 16, class 13 wins while the correct class is a close runner-up. Many trees make the same mistake after the central evidence is removed. Voting cannot repair a shared blind spot.

Table 3: Frozen tree and bagging model. Confidence is the winning class probability for the tree and the winning vote fraction for bagging.

Test set	Model	Accuracy	Confidence
Clean	Tree	53.75	96.25
	Bagging	85.00	41.13
Masked	Tree	18.75	100.00
	Bagging	68.75	27.18

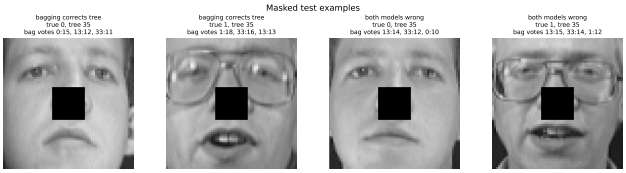


Figure 2: Two masked faces where bagging corrects the tree and two where both are wrong. The three largest nonzero vote counts are printed above each face.

3 Multiclass boosting from scratch

3.1 One complete SAMME round

I first checked the update on nine ordered points with three classes. A depth-one tree predicts $[1, 1, 1, 1, 1, 2, 2, 2, 2]$ for targets $[0, 0, 1, 1, 1, 2, 2, 2, 0]$. Three points are wrong, so $\epsilon = 1/3$ and

$$\alpha=\log\frac{1-1/3}{1/3}+\log(3-1)=1.3863. \tag{2}$$

The three misses receive four times the weight of a correct point after normalisation. The face-data code uses this same update. If a learner has $\epsilon \geq 1 - 1/K$, it is not added; if its error is numerically zero, the error is clipped for a finite weight and fitting stops.

3.2 Selecting depth and rounds

Table 5 gives clean validation accuracy for all 16 choices. Depth three with 50 rounds and depth four with 100 rounds both reach 35%. I chose 50 rounds because the validation score was already tied there. The frozen model has 72.08% training accuracy and 28.75% clean test accuracy.

At round 50, the current learner's weighted error is 0.7592, safely below the 40-class random threshold of 0.975. Its weight is 2.515. The largest image weight is 0.0253, and the ten heaviest images carry 0.1898 in total. The curve is not monotone because each new tree changes the distribution that the next tree sees.

3.3 Bagging against boosting

Bagging is slower because its trees are grown to full depth, but it is clearly better here. The 100-round

Table 4: The complete toy SAMME round. All initial weights are 1/9.

i	x	y	$h(x)$	Wrong	New weight
1	0	0	1	1	0.2222
2	1	0	1	1	0.2222
3	2	1	1	0	0.0556
4	3	1	1	0	0.0556
5	4	1	1	0	0.0556
6	5	2	2	0	0.0556
7	6	2	2	0	0.0556
8	7	2	2	0	0.0556
9	8	0	2	1	0.2222

Table 5: SAMME validation accuracy (percent).

Depth	10	25	50	100 rounds
1	3.75	8.75	13.75	8.75
2	7.50	13.75	18.75	12.50
3	15.00	17.50	35.00	28.75
4	10.00	16.25	28.75	35.00

boost also shows why rounds were selected on validation rather than training accuracy. Training accuracy continues upward while validation accuracy falls from the 50-round result.

The face in Figure 4 is misclassified in 45 of 50 rounds and reaches weight 0.0520. Its weight repeatedly rises after a miss and falls when a learner gets it right. That is useful when a rare but valid face needs attention. Here, repeated attention to a few difficult views also leaves too little capacity for the rest of the 40-way problem.

3.4 Response to the missing centre

For source face 6, bagging is correct on the clean image (class 0) but moves to class 13 after masking. Boosting predicts class 12 both times. Bagging's deep trees use many local pixel tests, so removing a central block changes several paths. The shallow boosted trees are less responsive to that block, but their stable answer is still wrong. Stability to a change is not useful by itself if the original rule is biased.

4 Bias and variance as measurements

4.1 Resampling protocol

I kept the 80 clean test faces fixed. For each of 30 repetitions, I sampled six positions with replacement from each identity's original six training faces. I then refit the frozen single tree, the selected 100-tree bagged model, and the selected 50-round boost without tuning again. For a test face, $p_r(x_i)$ is the 40-way class vector from repetition r. It is a tree probability, a bagging vote fraction, or a normalised SAMME weighted vote.

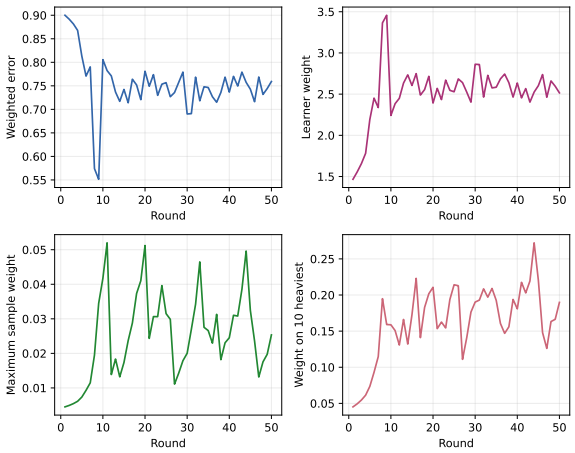


Figure 3: Selected SAMME run. The learner error and weight move from round to round as hard images receive more mass.

Table 6: Same learner counts for bagging and depth-three SAMME. Time is cumulative learner fit time in seconds on the run machine.

Model	T	Train	Val.	Test	Time
Bag	25	100	87.50	83.75	24.6
Boost	25	38.75	17.50	18.75	5.5
Bag	50	100	85.00	86.25	49.2
Boost	50	72.08	35.00	28.75	11.0
Bag	100	100	88.75	85.00	98.0
Boost	100	77.08	28.75	22.50	21.9

I computed

$$\text{Bias}^2 = \frac{1}{N}\sum_i \|\bar{p}(x_i) - e_i\|_2^2, \tag{3}$$

$$\text{Variance} = \frac{1}{NR}\sum_{i,r} \|p_r(x_i) - \bar{p}(x_i)\|_2^2. \tag{4}$$

The single tree has the smallest measured bias but by far the largest variance. Bagging raises bias by 0.0943, yet removes 0.6253 variance and raises mean accuracy by 32 points. Boosting is slightly steadier than bagging, but its bias is 0.9082. Its low variance means it repeats a weak rule consistently; it does not mean the rule is good.

4.2 What depth changes

I repeated the calculation at depth 2, depth 8, and unlimited depth. At each depth I also built a 100-tree bagged model. The leaf size stayed at the frozen value of two.

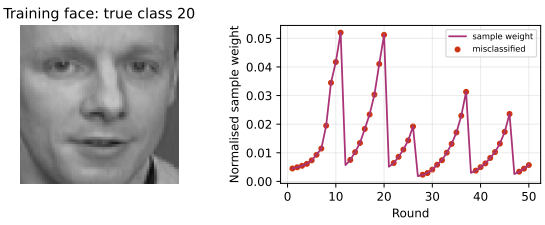


Figure 4: Training source index 204, class 20, receives the largest weight seen in the selected run. Red points mark rounds that misclassify it.

Table 7: Frozen models on clean and masked test sets. The last column is mean winning probability or vote fraction.

Set	Model	Accuracy	Win share
Clean	Tree	53.75	96.25
	Bagging	85.00	41.13
	Boosting	28.75	10.69
Masked	Tree	18.75	100.00
	Bagging	68.75	27.18
	Boosting	37.50	9.77

Table 9: Depth study over the same 30 resamples. L is the mean occupied leaf count. D is mean pairwise test disagreement among the 100 bagged trees.

Depth	Model	Bias^2	Var.	Accuracy	L	D
2	Tree	.9134	.0417	6.13	3.07	.9619
2	Bag	.9276	.0106	15.96	3.07	.9619
8	Tree	.7760	.1269	16.29	9.37	.9252
8	Bag	.8016	.0176	46.58	9.37	.9252
None	Tree	.3947	.6503	47.75	40.00	.8125
None	Bag	.4890	.0250	79.75	40.00	.8125

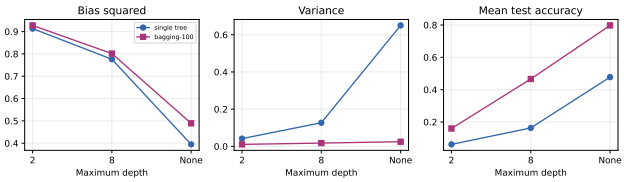


Figure 6: Bias squared, variance, and mean clean test accuracy as depth increases.

Depth lowers bias and opens more occupied leaves, but the fitted tree becomes more dependent on the sampled faces. Single-tree variance rises from 0.0417 at depth two to 0.6503 without a depth limit. Bagging removes 0.0310, 0.1093, and 0.6253 variance at the three depths. It helps little at depth two because averaging cannot create class distinctions that the three-leaf trees never learned. The singleton-leaf fraction is zero at all three depths because the frozen minimum leaf size is two.

The deepest setting gives the largest accuracy gain here: 32.00 points, compared with 30.29 at depth eight and 9.83 at depth two. That is a measured result for this split, not a rule that the deepest tree must always gain most. Its trees actually have lower pairwise disagreement (0.8125) than the shallower ensembles. The larger gain comes from combining low-bias trees whose individual predictions move strongly across resamples. Leaf capacity, disagreement, bias, and variance all matter together.



Figure 5: A test face where the two ensembles react differently to the mask.

Table 8: Measured quantities across 30 resampled training sets. Accuracy is mean ± sample standard deviation.

Model	Bias^2	Variance	Test accuracy
Tree	0.3947	0.6503	47.75 ± 4.55
Bagging	0.4890	0.0250	79.75 ± 2.73
Boosting	0.9082	0.0185	17.38 ± 4.85

4.3 Final comparison

Table 10: The three selected models. Clean and masked columns use the single fixed split; bias and variance use the 30 resampled fits.

Model	Clean	Masked	Bias^2	Variance
Tree	53.75	18.75	0.3947	0.6503
Bagging	85.00	68.75	0.4890	0.0250
Boosting	28.75	37.50	0.9082	0.0185

Bagging is the most stable useful model: its variance is close to boosting's, but its clean accuracy is much higher. The tree is most sensitive to the missing region, losing 35 points. Bagging loses 16.25 points. Boosting rises by 8.75 points after masking, but both of its scores are low; this is not meaningful resistance to missing data. The lowest-variance model is therefore not the model with the best test accuracy. Bias can dominate, as it does for boosting here.

5 Reproduction notes

The archive contains the exact split in CSV and NPZ form, the JSON configuration, every table used here, plot files, the final run log, and the complete source. Running

> python src/run_experiments.py –repetitions 30 –jobs 8

from the repository root rebuilds the results. Parallel work is only across independent bootstrap fits or independent repetitions. Every random draw has an explicit seed, so changing the worker count does not change the saved numbers.

6 Conclusion

The main result is quite concrete. A deep tree is useful for 40-way face recognition, but its exact rules move a lot when the six training images per person are resampled. Bagging keeps the useful depth and averages away much of that movement. Shallow trees are steady but cannot separate enough identities, so bagging cannot rescue them. The selected SAMME model is also steady, but it spends too much weight on a small set of difficult faces and retains high bias. On this split, the 100-tree bagged model is the best fit to both the clean task and the missing-centre test.

References

[1] J. Zhu, H. Zou, S. Rosset, and T. Hastie. Multi-class AdaBoost. Statistics and Its Interface, 2009.

[2] F. S. Samaria and A. C. Harter. Parameterisation of a stochastic model for human face identification. Proceedings of the IEEE Workshop on Applications of Computer Vision, 1994.